

ASG TASK

1. Create one VPC in N. Virginia region.

Step 1 — Create VPC

1. Login to **AWS Console**
2. Go to **VPC**
3. Click **Create VPC**

Configuration:

Setting	Value
---------	-------

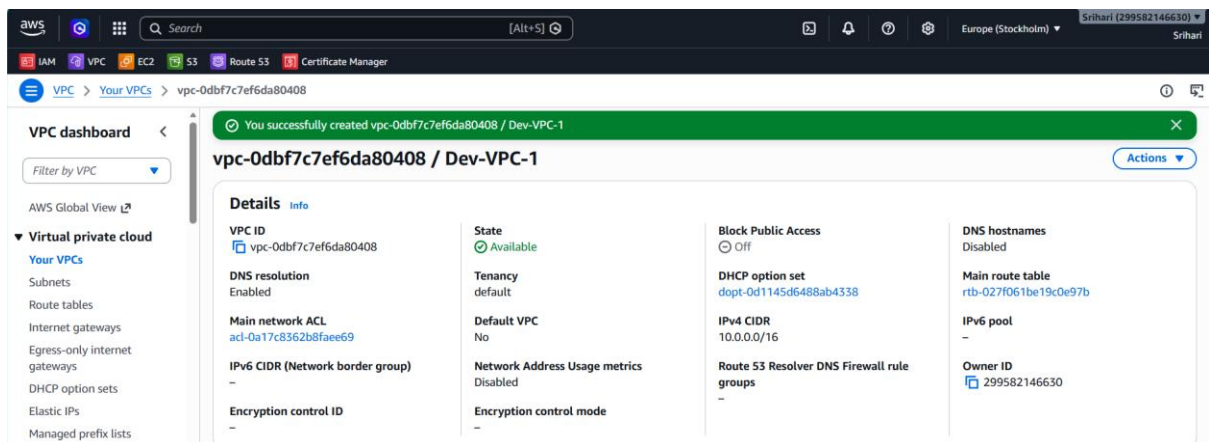
Name	Dev-VPC-1
------	-----------

IPv4 CIDR	10.0.0.0/16
-----------	-------------

Tenancy	Default
---------	---------

Region	N. Virginia (us-east-1)
--------	-------------------------

Click **Create**



2. Create two subnets: one public subnet and one private subnet.

Step 2 — Create Subnets

Go to **Subnets** → **Create subnet**

Public Subnet

Setting Value

Name	Public-Subnet
------	---------------

VPC	Dev-VPC
-----	---------

CIDR 10.0.1.0/24

Enable **Auto assign Public IP**

The screenshot shows the AWS Management Console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and a list of services (IAM, VPC, EC2, S3, Route 53, Certificate Manager). The user is logged in as 'Srihari' from 'Europe (Stockholm)'. The main content area is titled 'VPC dashboard' and 'Subnets (1/4)'. It features a search bar for 'Find subnets by attribute or tag'. Below this is a table with columns: Name, Subnet ID, State, VPC, Block Public..., and IPv4 CIDR. The table lists three subnets, all in an 'Available' state, associated with the 'vpc-0e8e7ec4e64e7c5cb' VPC. The first subnet is 'Public-Subnet' with ID 'subnet-0b12f659d3f5c3d0c' and CIDR '10.0.1.0/24'. The other two subnets have IDs 'subnet-079d3e04543e35e9' and 'subnet-07f5ff49850fb8795'. The 'Public-Subnet' is selected with a blue checkmark in the first column.

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
☐ -	subnet-079d3e04543e35e9	Available	vpc-0e8e7ec4e64e7c5cb	Off	172.31.16.0/
☐ -	subnet-07f5ff49850fb8795	Available	vpc-0e8e7ec4e64e7c5cb	Off	172.31.32.0/
☐ -	subnet-0e8cf5ed90423645d	Available	vpc-0e8e7ec4e64e7c5cb	Off	172.31.0.0/2
☒ Public-Subnet	subnet-0b12f659d3f5c3d0c	Available	vpc-0dbf7c7ef6da80408 Dev:...	Off	10.0.1.0/24

The screenshot shows the AWS Management Console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and a user profile for 'Srihari'. Below the navigation bar is a breadcrumb trail: 'VPC > Subnets > subnet-Ob12f659d3f5c3d0c > Edit subnet settings'. The main heading is 'Edit subnet settings' with an 'info' link. The page is divided into two sections. The first section, 'Subnet', shows the 'Subnet ID' as 'subnet-Ob12f659d3f5c3d0c' and the 'Name' as 'Public-Subnet'. The second section, 'Auto-assign IP settings', has a description: 'Enable AWS to automatically assign a public IPv4 or IPv6 address to a new primary network interface for an instance in this subnet.' It contains two checkboxes: 'Enable auto-assign public IPv4 address' (checked) and 'Enable auto-assign customer-owned IPv4 address' (unchecked). A note below the second checkbox states: 'Option disabled because no customer owned pools found.'

Private Subnet

Setting Value

Name	Private-Subnet
------	----------------

VPC Dev-VPC

AZ us-east-1a

CIDR 10.0.2.0/24

✔ You have successfully created 1 subnet: subnet-08b97079f4c9c50bd
✕

Subnets (1/1) Info

Subnet ID : subnet-08b97079f4c9c50bd ✕

Clear filters

Last updated less than a minute ago 🔄
Actions ▾
Create subnet

☒	Name ▾	Subnet ID ▾	State ▾	VPC ▾	Block Public... ▾	IPv4 CIDR
☒	private-subnet	subnet-08b97079f4c9c50bd	✔ Available	vpc-0dbf7c7ef6da80408 Dev...	☹ Off	10.0.2.0/24

3. Attach an IGW to the VPC.

Step 3 — Create Internet Gateway

Go to **Internet Gateway**

Create new:

| Name | Dev-IGW |

After creating:

Attach IGW to VPC

Actions → Attach → Select **Dev-VPC**

The screenshot shows the AWS Management Console interface for an Internet Gateway. The top navigation bar includes 'Route 53' and 'Certificate Manager'. The breadcrumb trail shows 'igw-0ab1c4ad7615f9fab'. A green notification banner at the top states: 'The following internet gateway was created: igw-0ab1c4ad7615f9fab - Dev-IGW. You can now attach to a VPC to enable the VPC to communicate with the internet.' with an 'Attach to a VPC' button.

The main heading is 'igw-0ab1c4ad7615f9fab / Dev-IGW' with an 'Actions' dropdown menu.

Details [Info](#)

Internet gateway ID	State	VPC ID	Owner
igw-0ab1c4ad7615f9fab	Detached	-	299582146630

Tags (1) [Manage tags](#)

Key	Value
Name	Dev-IGW

[Alt+S] Europe (Stockholm) Srihari (299582146630) Srihari

Route 53 Certificate Manager

igw-0ab1c4ad7615f9fab

Internet gateway igw-0ab1c4ad7615f9fab successfully attached to vpc-0dbf7c7ef6da80408

igw-0ab1c4ad7615f9fab / Dev-IGW [Actions](#)

Details [Info](#)

Internet gateway ID	State	VPC ID	Owner
igw-0ab1c4ad7615f9fab	Attached	vpc-0dbf7c7ef6da80408 Dev-VPC-1	299582146630

4. Create one public route table (RT) and one private route table.

Step 4 — Create Route Tables

Public Route Table

Go to **Route Tables** → **Create**

| Name | Public-RT |

| VPC | Dev-VPC |

The screenshot shows the AWS Management Console interface for a route table named 'Public-RT' (ID: rtb-071cb1d7286192efd). A green notification bar at the top states 'Route table rtb-071cb1d7286192efd | Public-RT was created successfully.' The main content area displays the route table's details, including its ID, VPC (vpc-0dbf7c7ef6da80408 | Dev-VPC-1), and owner ID (299582146630). Below the details, there are tabs for 'Routes', 'Subnet associations', 'Edge associations', 'Route propagation', 'Tags', and 'Related resources'. The 'Routes' tab is selected, showing a table with one route. The route has a destination of 10.0.0.0/16, a target of 'local', and a status of 'Active'. The 'Propagated' column shows 'No' and the 'Route Origin' is 'Create Route Table'.

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	Create Route Table

Edit Routes:

Add:

Destination : 0.0.0.0/0

Target : Internet Gateway (Dev-IGW)

The screenshot shows the 'Edit routes' interface for the 'Public-RT' route table. It displays a table with two routes. The first route has a destination of 10.0.0.0/16, a target of 'local', and a status of 'Active'. The second route has a destination of 0.0.0.0/0, a target of 'Internet Gateway', and a status of '-'. The 'Propagated' column shows 'No' for both routes, and the 'Route Origin' is 'CreateRouteTable'. There is a 'Remove' button next to the second route. At the bottom, there are buttons for 'Add route', 'Cancel', 'Preview', and 'Save changes'.

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
0.0.0.0/0	Internet Gateway	-	No	CreateRouteTable

Associate:

Subnet → Public-Subnet

VPCRoute tablesrtb-071cb1d7286192efdEdit subnet associations

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (1/2)

Filter subnet associations

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
private-subnet	subnet-08b97079f4c9c50bd	10.0.2.0/24	-	Main (rtb-027f061be19c0e97b)
Public-Subnet	subnet-0b12f659d3f5c3d0c	10.0.1.0/24	-	Main (rtb-027f061be19c0e97b)

Selected subnets

subnet-0b12f659d3f5c3d0c / Public-Subnet

Cancel

Save associations

Private Route Table

Create another:

| Name | Private-RT |

b-03aeff26834703198

Route table rtb-03aeff26834703198 | Private-RT was created successfully.

rtb-03aeff26834703198 / Private-RT

Actions

Details

Route table ID
rtb-03aeff26834703198

VPC
vpc-0dbf7c7ef6da80408 | Dev-VPC-1

Main
No

Owner ID
299582146630

Explicit subnet associations
-

Edge associations
-

Routes

Subnet associations

Edge associations

Route propagation

Tags

Related resources

Routes (1)

Both

Edit routes

Filter routes

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	Create Route Table

Associate:

Subnet → Private-Subnet

aws

Search

[Alt+S]

Europe (Stockholm)

Srihari (299582146630)

Srihari

IAM

VPC

EC2

S3

Route 53

Certificate Manager

VPCRoute tablesrtb-03aeff26834703198Edit subnet associations

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (1/2)

Filter subnet associations

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
private-subnet	subnet-08b97079f4c9c50bd	10.0.2.0/24	-	Main (rtb-027f061be19c0e97b)
Public-Subnet	subnet-0b12f659d3f5c3d0c	10.0.1.0/24	-	rtb-071cb1d7286192efd / Public-RT

Selected subnets

subnet-08b97079f4c9c50bd / private-subnet

Cancel

Save associations

5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

Step 5 — Create NAT Gateway

Go to NAT Gateway

Create NAT:

Setting	Value
Name	Dev-NAT
Subnet	Public-Subnet
Elastic IP	Allocate new

Click Create

[Alt+S] ⓘ

ⓧ ⓘ ⚙ Europe (Stockholm) ▾ Srihari (299582146630) ▾ Srihari

Route 53 Certificate Manager

ⓘ ⓧ

ⓧ NAT gateway nat-19270698f699f97d3 | Dev-NAT was created successfully.

nat-19270698f699f97d3 / Dev-NAT

Actions ▾

Details

NAT gateway ID

nat-19270698f699f97d3

NAT gateway ARN

arn:aws:ec2:eu-north-1:299582146630:natgateway/nat-19270698f699f97d3

VPC

vpc-0dbf7c7ef6da80408 / Dev-VPC-1

Availability mode

Regional

Connectivity type

Public

Method of EIP allocation

Automatic

State

⌚ Pending

Created

Wednesday, July 29, 2026 at 20:07:54 GMT+5:30

State message

Info

–

Deleted

–

Attach NAT to Private Route Table

Go to Private-RT

Edit Routes

Add:

Destination : 0.0.0.0/0
Target : NAT Gateway

Edit routes: rtb-03aeff26834703198 / Private-RT

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
0.0.0.0/0	NAT Gateway	-	No	CreateRoute

Buttons: Add route, Cancel, Preview, Save changes

Now Private subnet can access internet for updates

rtb-03aeff26834703198 / Private-RT

Details

- Route table ID: rtb-03aeff26834703198
- VPC: vpc-0dbf7c7ef6da80408 | Dev-VPC-1
- Main: No
- Owner ID: 299582146630
- Explicit subnet associations: subnet-08b97079f4c9c50bd / private-subnet
- Edge associations: -

Routes (2)

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	nat-19270698f699f97d3	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

6. Create two instances, one in the public subnet and one in the private subnet.

Step 6 — Launch EC2 Instances

Go to EC2 → Launch Instance

Public EC2

Configuration:

Setting	Value
Name	Public-Server
AMI	Amazon Linux 2
Type	t2.micro
Subnet	Public-Subnet
Auto Public IP	Enable
Security Group:	

Allow:

HTTP 80

SSH 22

[Alt+S]

Europe (Stockholm)

Srihari (299582146630)

Srihari

Route 53

Certificate Manager

Instances (1/2) Info

Last updated 3 minutes ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

< 1 >

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☐	AmazonLinux-...	i-08e9f6db56279dace	Stopped	t3.micro	-	eu-north-1b	-
☒	Public-Server	i-0cca08e1d95a619e8	Running	t3.micro	Initializing	eu-north-1a	ec2-13-61-148-100.eu-...

Private EC2

Launch another:

Setting Value

Name Private-Server

AMI Amazon Linux 2

Subnet Private-Subnet

Public IP Disable

Security group:

Allow:

HTTP 80

SSH from Public Instance Security Group

Europe (Stockholm)

Srihari (299582146630)

Srihari

Route 53

Certificate Manager

Instances (1/3) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

< 1 >

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☐	AmazonLinux-...	i-08e9f6db56279dace	Stopped	t3.micro	-	eu-north-1b	-
☒	private-server	i-0a6d4f113fe6dd436	Running	t3.small	Initializing	eu-north-1b	-
☐	Public-Server	i-0cca08e1d95a619e8	Running	t3.micro	3/3 checks passed	eu-north-1a	ec2-13-61-148-100.eu-...

7. Deploy Apache server on both EC2 instances with a sample index.html file.

SSH into **Public Instance**

Run:

```
sudo yum update -y
```

```
sudo yum install httpd -y
```

```
sudo systemctl start httpd
```

```
sudo systemctl enable httpd
```

```
[ec2-user@ip-10-0-1-125 ~]$ sudo su
[root@ip-10-0-1-125 ec2-user]# sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-10-0-1-125 ec2-user]# sudo yum install httpd -y
Last metadata expiration check: 0:00:12 ago on Thu Jul 30 06:28:45 2026.
Dependencies resolved.
=====
Package                                Architecture                               Version
=====
Installing:
httpd                                  x86_64                                    2.4.68-1.amzn2023.0.1
Installing dependencies:
apr                                    x86_64                                    1.7.5-1.amzn2023.0.4
apr-util                              x86_64                                    1.6.3-1.amzn2023.0.2
apr-util-ldap                         x86_64                                    1.6.3-1.amzn2023.0.2
generic-logos-httpd                  noarch                                    18.0.0-12.amzn2023.0.3
httpd-core                           x86_64                                    2.4.68-1.amzn2023.0.1
httpd-filesystem                     noarch                                    2.4.68-1.amzn2023.0.1
httpd-tools                          x86_64                                    2.4.68-1.amzn2023.0.1
libbrotli                             x86_64                                    1.0.9-4.amzn2023.0.2
mailcap                              noarch                                    2.1.49-3.amzn2023.0.3
Installing weak dependencies:
apr-util-openssl                     x86_64                                    1.6.3-1.amzn2023.0.2
mod_http2                            x86_64                                    2.0.42-1.amzn2023.0.1
mod_lua                              x86_64                                    2.4.68-1.amzn2023.0.1
Transaction Summary

```

Create sample page:

```
sudo nano /var/www/html/index.html
```

Example:

```
<h1>Public Server</h1>
```

```
[root@ip-10-0-1-125 ec2-user]# sudo systemctl start httpd
[root@ip-10-0-1-125 ec2-user]# sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@ip-10-0-1-125 ec2-user]# sudo nano /var/www/html/index.html
[root@ip-10-0-1-125 ec2-user]#
```



Now SSH into **Private** instance via **Public** instance (Bastion)

```
SRIHARI@LAPTOP-N3TTSS9I MINGW64 ~/Downloads
$ cat awskey.pem
-----BEGIN RSA PRIVATE KEY-----
MIIEpQIBAAKCAQEAzSiVG4k0y6uKG9czcNuy3tAWUhLqhZL5pwkod0duv34nk2wg
aF4ehUgzxLOIHEyUYvbHVT2RdeRIiJADmf9fwukT3e1tZ/izzXhrgy41/7uy5iN0
g52Yp9Ck6f2npc0IZcPD6g9yiU8BL/5g6bSqye0Z1tWmUisLiHb0JTrzwiDB6Jhb
1xDTEdNUdwdau+M+AzrQCBxkAAaQvH0J1vHiU19YaaUaL55YZcQLbJ+kSiNLMUGp
SeJB7yj8Ajxm1Syf7FGnR3Awwq1nsPuHGnpbwuuHpkUHZXiVYXeg3WPstGTS4cc6
/0dJIGFrss0fRbJXLImH1ERSilVvZDSqLcj/4QIDAQABaoIBAA9LlUJGR5oboeJN
ewJQ/tbpwKBB1MY8n0w9HI04AC06rgyppQji63eYWH0zxtQFSZLPPzE1s+CMMnQr
+HjEt0E520dSHdwAiLcq4SkYiuIUBlRuFE0zviUvFbNmHADuc/0dmi+2hUu1LIV
TvhhdYjz9H8/7dFhoi3CwC1MnMoh+BNEn4P/j4+Cz0H13a1y8jywPg/wXTDtzFh
igAZEgB2udTs4RVQD/5UN/FnYUk1JUionv2JUraq0+kHmS5ZaZ1zzKz4v/Xn93Uy
7xaSfdw2dxXrKvruFzT0syLKrLrtCxo00GEECb5sy0ocId2jtUKnkfX2nC6X7Qf/
Dj8DiSECgYEA5k1FHEuFRI3gLOxkqUdMfSTdQWXdMDz4l2wKYlt71gtsM6vjWPef
5kv5DXWh3Kw8j61nEhE7wRkL1M8c7PIOzn+90TuXFRzGxPAuMx565pKAJCz91i7v
y/3FOzvvNodcyGHibGnmZ9ALVBPUpitK1bwVOagBQ8wdCYexPo1cI00CgYEA5A0T
mX/jWqBhigj/TQnY9Ef2+QwLxCJZ8sKGYxLuGTDEtHA7NHooTiM2fdPRUCBXWuo8
6Bw8cYmkTQk7p0N9zoytDHJ3gBdAxRABYYW0EVB4IYq0FmG7LUVq7wk1B1xNnp0i
Zm6Vp3Inr4L4EXqEsdu3z0wxcTGxPU/ht84FHOUcGyEA0qTkTIAK0exOnBRskaRk
IqagNNvd1uw9mAo66UGx39Is0ooqdMpZT443qwn2HY1I11BR/Y5SGIHP0t8soCn
6FpQEdQewTIR4B+qshjz4ocm62+3cLsXm+ZuCsLEBoUC6Ukw14C5PES+x9UnkCFS
3nGcnFK9bwX5Nq1FBeI32JUCgYEA58V7UN6pioE0B5DPV7JLPgjjV7XYgy077hfE
uLWngY1wMKgcMWH0WNj1ruQbpTy9GNfXRfzQm016281C38IyninQL6JmeVGUhuKn
k9eiG07e5p3EJHAK3cn9RwvBqkqefx52WhruQDQPyVjiUJht07mQSAMzpJN8DyoI
ju9xfz0CgYEA5Kas7/Pzw4FV6WbSxAA9r7pYJt1pPwEtZsWM3JRMBSATHKLAR4
nS+gxDBSmRnT0izj130g9g1trymKOMBgk/Zit8WeGWG5fG5Vs tnb1r1J2fHxznkm
MepfYwUm1ZKoO+Kf1L07jX9yiSEHvmTpttpofRZj9wrHzXtm0j3/2Jg=
-----END RSA PRIVATE KEY-----
```

Ssh-l pemkey ec2-user@private-ip

Install Apache:

```
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
```


8. Create one application load balancer and attach it to both EC2 instances.

Step 8 — Create Application Load Balancer

Go to **EC2** → **Load Balancer**

Create **Application Load Balancer**

Configuration:

Setting	Value
Name	Dev-ALB
Type	Internet Facing
VPC	Dev-VPC
Subnets	Public-Subnet

Add two public subnets

Listener:

HTTP : 80

NOTE : Issue: Internet-facing ALB Creation Failed

Problem:

While creating an Internet-facing Application Load Balancer (ALB), AWS did not allow the selected subnet configuration.

Cause:

An Internet-facing ALB requires at least two public subnets in different Availability Zones (AZs). Initially, only one public subnet was available in the VPC, preventing the ALB from being configured correctly.

Resolution:

- Created an **additional public subnet** in a different Availability Zone.
- Associated the new subnet with the public route table containing the Internet Gateway (IGW) route (0.0.0.0/0).
- Selected both public subnets while creating the Internet-facing ALB.
- Successfully created the ALB and attached the required Target Group.

Result:

The Internet-facing Application Load Balancer was created successfully and became accessible using its public DNS name.

[Alt+S]

Europe (Stockholm)

Srihari (299582146630)

Route 53

Certificate Manager

dev-alb-1

dev-alb-1

Actions

▼ Details

Load balancer type

Application

Scheme

Internet-facing

Status

Active

Hosted zone

Z23TAZ6LKFMNIO

VPC

[vpc-0dbf7c7ef6da80408](#)

Availability Zones

[subnet-0685d128690f1a054](#) eu-north-1c (eun1-az3)
[subnet-0b12f659d3f5c3d0c](#) eu-north-1a (eun1-az1)

Load balancer IP address type

IPv4

Date created

July 30, 2026, 15:28 (UTC+05:30)

Load balancer ARN

[arn:aws:elasticloadbalancing:eu-north-1:299582146630:loadbalancer/app/dev-alb-1/aa847752681ca379](#)

DNS name

[dev-alb-1-1488807343.eu-north-1.elb.amazonaws.com](#) (A Record)

Create Target Group

Setting	Value
Name	Apache-TG
Target Type	Instance
Protocol	HTTP
Port	80

Register targets:

Public-Server

Private-Server

1.console.aws.amazon.com/ec2/home?region=eu-north-1#TargetGroup:targetGroupArn=arn:aws:elasticloadbalancing:eu-north-1:299582146630:targetgroup/apache-T...

[Alt+S]

Europe (Stockholm)

Srihari (299582146630)

Route 53

Certificate Manager

ache-TG

apache-TG

Actions

Details

[arn:aws:elasticloadbalancing:eu-north-1:299582146630:targetgroup/apache-TG/2a719d57cbf76dde](#)

Target type

Instance

Protocol : Port

HTTP: 80

Protocol version

HTTP1

VPC

[vpc-0dbf7c7ef6da80408](#)

IP address type

IPv4

Load balancer

[dev-alb-1](#)

Target access analysis

Access issue detected

2

Total targets

1

Healthy

0

Unhealthy

1

Unused

0

Initial

0

Draining

0

Anomalous

► Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Now access:

ALB DNS Name:

DEV-ALB-1-1488807343.us-north-1.elb.amazonaws.com

Traffic will load balance between servers.

Public Server

9. Store application load balancer logs in S3.

Step 9 — Store ALB Logs in S3

Go to S3

Create bucket:

alb-logs-devops

give bucket policy

The screenshot shows the AWS Management Console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and the region 'Europe (Stockholm)'. Below the navigation bar, the breadcrumb trail shows 'Amazon S3 > Buckets > alb-logs-devops-01'. A green notification banner at the top of the console area states 'Successfully created bucket "alb-logs-devops-01"'. The main content area shows the bucket 'alb-logs-devops-01' with tabs for 'Objects', 'Metadata', 'Properties', 'Permissions', 'Metrics', 'Management', 'File systems - new', and 'Access Points'. The 'Objects' tab is selected, showing 'Objects (0)' and a message: 'Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)'. Below this is a search bar and a table with columns: Name, Type, Last modified, Size, and Storage class. The table is empty, with a message: 'No objects. You don't have any objects in this bucket.' and an 'Upload' button. A second green notification banner at the bottom of the console area states 'Successfully edited bucket policy.'. Below the console area, there's a section titled 'Bucket policy' with a description: 'The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other account'. A code editor shows the following JSON policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AWSLogDeliveryWrite",
      "Effect": "Allow",
      "Principal": {
        "Service": "logdelivery.elasticloadbalancing.amazonaws.com"
      },
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::alb-logs-devops-01/AWSLogs/299582146630/*"
    }
  ]
}
```

Go to **Load Balancer → Attributes**

Enable:

Access logs → Enabled

Bucket → alb-logs-devops

Logging

All legacy logs are stored in S3. If no prefix is specified, logs are stored in the bucket root.

Access logs - legacy

Access logs deliver detailed logs of all requests made to your load balancer. Additional charges apply. [Learn more](#)

S3 URI

Q

s3://alb-logs-devops-01

X

View

Browse S3

Format: s3://bucket/prefix

☐ Connection logs - legacy

Connection logs deliver detailed logs of all connections made to your load balancer. Additional charges apply. [Learn more](#)

☐ Health check logs - legacy

Health check logs deliver detailed logs of health checks for all targets in your load balancer's target groups. Additional charges apply. [Learn more](#)

Logging options

Log delivery to CloudWatch, Kinesis Firehose, and S3 is configurable through an integration with Logging - by Amazon CloudWatch. Visit **Logging** in the [Integrations tab](#).

Cancel

Save changes

Successfully modified load balancer attributes.

X

dev-alb-1

Actions

▼ Details

Load balancer type

Application

Scheme

Internet-facing

Status

Active

Hosted zone

Z23TAZ6LKFMNIO

VPC

[vpc-0dbf7c7ef6da80408](#)

Availability Zones

[subnet-0685d128690f1a054](#) eu-north-1c (eu1-az3)

[subnet-0b12f659d3f5c3d0c](#) eu-north-1a (eu1-az1)

Load balancer IP address type

IPv4

Date created

July 30, 2026, 15:28 (UTC+05:30)

Load balancer ARN

[arn:aws:elasticloadbalancing:eu-north-1:299582146630:loadbalancer/app/dev-alb-1/aa847752681ca379](#)

DNS name

[dev-alb-1-1488807343.eu-north-1.elb.amazonaws.com](#) (A Record)

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Attributes

Capacity

Tags

10.Store the VPC flow logs in a CloudWatch log group.

Step 10 — Enable VPC Flow Logs

Go to **VPC → Flow Logs**

Create:

Setting Value

Resource Dev-VPC

Destination CloudWatch

Log Group vpc-flow-logs

Create flow log [Info](#)

Flow logs can capture IP traffic flow information for the network interfaces associated with your resources. You can create multiple flow logs to send traffic to different destinations.

Selected resources [Info](#)

Flow logs will only be created for resources in an available state.

Name	Resource ID	State
Dev-VPC-1	vpc-0dbf7c7ef6da80408	Available

Flow log settings

Name - optional

my-vpc-flowlogs

Filter

The type of traffic to capture (accepted traffic only, rejected traffic only, or all traffic).

☐ Accept
☐ Reject
☒ All

Maximum aggregation interval [Info](#)

The maximum interval of time during which a flow of packets is captured and aggregated into a flow log record.

☒ 10 minutes
☐ 1 minute

Destination

The destination to which to publish the flow log data.

☒ Send to CloudWatch Logs
☐ Send to an Amazon S3 bucket

CloudWatch

Never expire

Stored bytes: 3.89 MB

Metric filters: 0

Data protection: -

Sensitive data count: -

Syslog ingestion [Info](#) [Configure](#)

Log streams | Tags | Data protection | Anomaly detection | Metric filters | Subscription filters | Contributor Insights | Field indexes | Transformer

Log streams (6)

By default, we only load the most recent log streams.

Filter log streams or try prefix search

☐ Exact match ☐ Show expired [Info](#)

Log stream	Last event time
eni-0585f23d235235ec4-all	2026-07-30 13:25:22 (UTC)
eni-007c1b06282b09a87-all	2026-07-30 13:25:08 (UTC)
eni-041afceeb76227a26-all	2026-07-30 13:25:03 (UTC)
nat-19270698f699f97d3-all	2026-07-30 13:24:45 (UTC)
nat-15c43dad4b3b77966-all	2026-07-17 11:47:32 (UTC)
eni-07b78d78d0570eb77-all	2026-07-16 12:11:40 (UTC)

11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

Step 11 — Create CloudWatch Dashboard

Go to **CloudWatch** → **Dashboard**

Create dashboard:

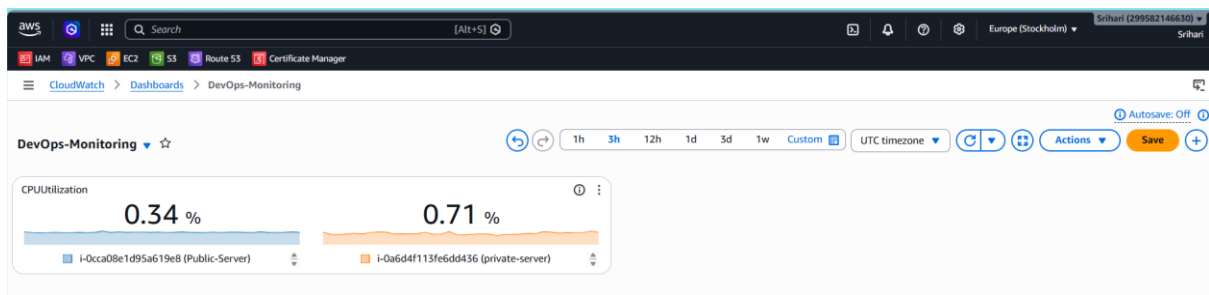
DevOps-Monitoring

Add widget:

CPU Utilization

Metric → **EC2** → **Per Instance** → **CPUUtilization**

Add both instances.



Apache Monitoring

Install CloudWatch agent.

SSH into instance:

```
[ec2-user@ip-192-168-0-234 ~]$ sudo yum update -y
Last metadata expiration check: 1:54:47 ago on Thu Jul 30 09:43:15 2026.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-192-168-0-234 ~]$ sudo yum install -y amazon-cloudwatch-agent
Last metadata expiration check: 1:54:59 ago on Thu Jul 30 09:43:15 2026.
Dependencies resolved.
```

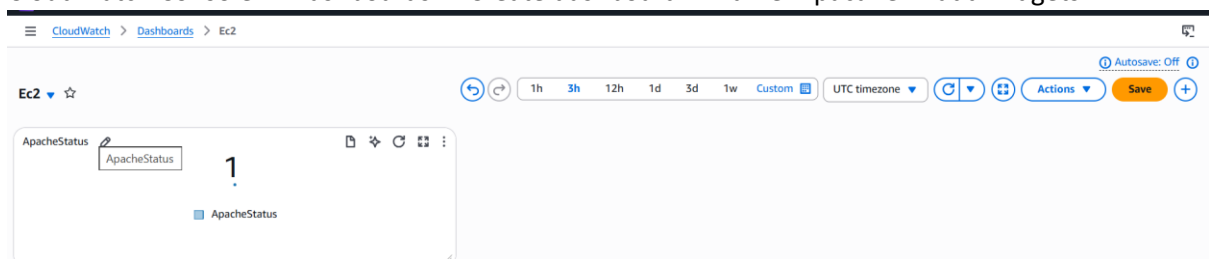
Package	Architecture	Version	Repository	Size
Installing:				
amazon-cloudwatch-agent	x86_64	1.300067.1-1.amzn2023	amazonlinux	68 M
Transaction Summary				
Install 1 Package				
Total download size: 68 M				
Installed size: 231 M				
Downloading Packages:				
amazon-cloudwatch-agent-1.300067.1-1.amzn2023.x86_64.rpm			68 MB/s 68 MB	00:01
Total			65 MB/s 68 MB	00:01
Running transaction check				
Transaction check succeeded.				
Running transaction test				
Transaction test succeeded.				
Running transaction				
Preparing :				

1/1

```
[ec2-user@ip-192-168-0-234 ~]$ sudo systemctl status amazon-cloudwatch-agent
● amazon-cloudwatch-agent.service - Amazon CloudWatch Agent
   Loaded: loaded (/etc/systemd/system/amazon-cloudwatch-agent.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-07-30 11:45:06 UTC; 12s ago
     Main PID: 34261 (amazon-cloudwat)
       Tasks: 7 (limit: 1059)
      Memory: 47.0M
         CPU: 316ms
    CGroup: /system.slice/amazon-cloudwatch-agent.service
            └─34261 /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -config /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml -envconfig /opt/aws/

Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/am
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json does not exist or cannot
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/am
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 !! Valid json input schema.
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: !! Trying to detect region from ec2
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 D! ec2tagger processor required because append dimensions is set
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 D! delta processor required because metrics with diskio or net are set
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 D! ec2tagger processor required because append dimensions is set
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 Configuration validation first phase succeeded
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34261]: !! Detecting run as user...
```

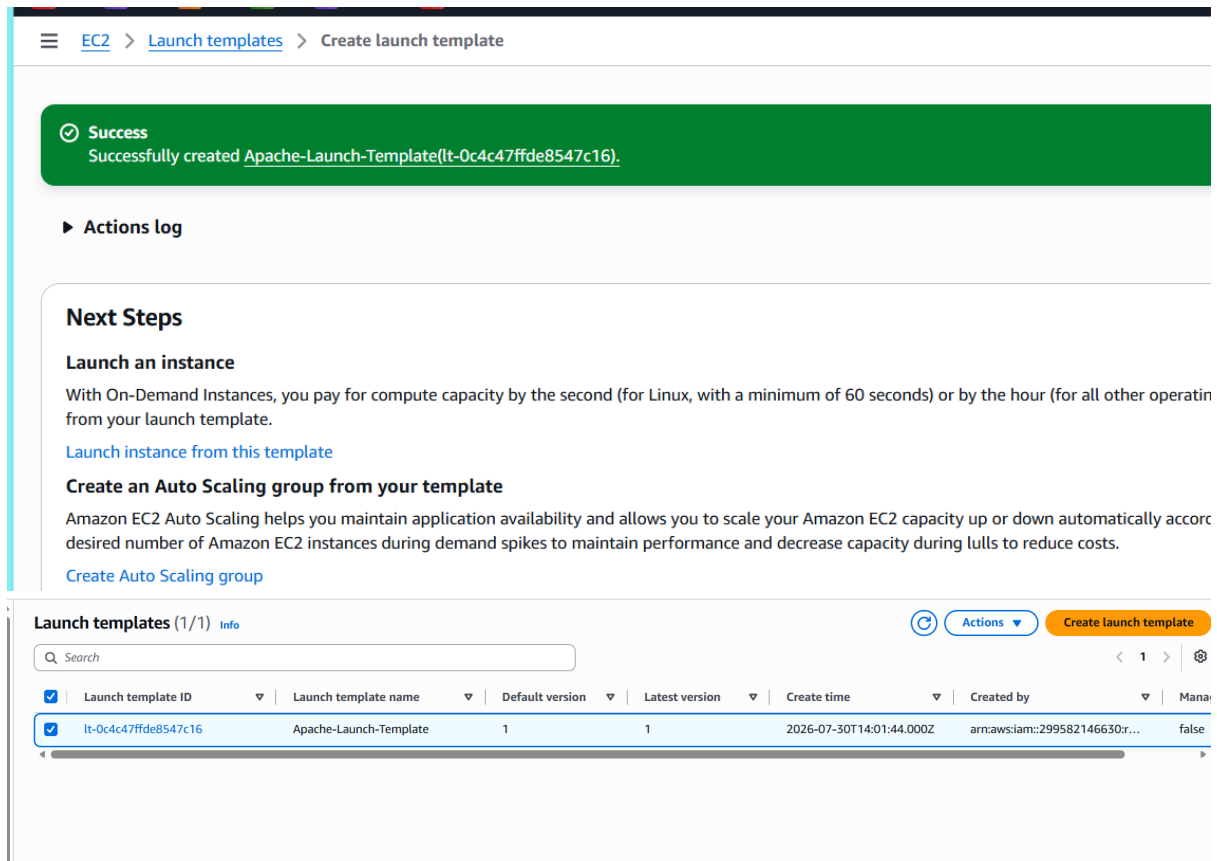
CloudWatch Console -> Dashboards -> Create dashboard -> name Apacache -> add widgets:



12.If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

Step 1: Create a Launch Template

EC2 Console -> Launch Templates -> Create launch template -> configure using lab-public-ec2's AMI, instance type, key pair, security group (lab-web-sg), and subnet settings:



The screenshot shows the AWS Management Console interface. At the top, the breadcrumb navigation reads 'EC2 > Launch templates > Create launch template'. A green success banner at the top states: 'Success Successfully created Apache-Launch-Template(lt-0c4c47ffde8547c16)'. Below this is an 'Actions log' section. The 'Next Steps' section includes links for 'Launch an instance' and 'Create an Auto Scaling group from your template'. At the bottom, the 'Launch templates (1/1)' list is displayed with a table containing one entry.

Launch template ID	Launch template name	Default version	Latest version	Create time	Created by	Managed by
lt-0c4c47ffde8547c16	Apache-Launch-Template	1	1	2026-07-30T14:01:44.000Z	arn:aws:iam::299582146630r...	false

Step 2: Create the Auto Scaling Group

EC2 Console -> Auto Scaling Groups -> Create Auto Scaling group -> configure:

EC2

>

Auto Scaling groups

>

Create Auto Scaling group

Step 1

Choose launch template

Step 2

Choose instance launch options

Step 3 - optional

Integrate with other services

Step 4 - optional

Configure group size and scaling

Step 5 - optional

Add notifications

Step 6 - optional

Add tags

Step 7

Review

Choose launch template

Info

Specify a launch template that contains settings common to all EC2 instances that are launched by this Auto Scaling group.

Name

Auto Scaling group name

Enter a name to identify the group.

Test-ASG

Must be unique to this account in the current Region and no more than 255 characters.

Launch template

Info

For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.

Launch template

Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.

My-ASG

Create a launch template

Version

Default (1)

Create a launch template version

Description

-

Launch template

My-ASG

lt-0b5b261bf94a3bed9

Instance type

t3.micro

AMI ID

ami-02b64aa047cb5edf5

Security groups

-

Request Spot Instances

No

Key pair name

multitool-key

Security group IDs

sg-0e095614194496251

Additional details

Storage (volumes)

-

Date created

Thu Jul 30 2026 18:03:02 GMT+0530 (India Standard Time)

Network

Info

For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.

VPC

Choose the VPC that defines the virtual network for your Auto Scaling group.

vpc-0ea200865b2aa88e0 (My-vpc)

192.168.0.0/20

Create a VPC

Availability Zones and subnets

Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

Select Availability Zones and subnets

use1-az1 (us-east-1a) | subnet-049df76aaa0186157 (public-subnet)

192.168.0.0/24

Create a subnet

Availability Zone distribution - new

Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.

Balanced best effort

If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.

Balanced only

If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.

Reservations Then Balanced

Prioritizes launching into Capacity Reservations, distributing across AZs with available reservations. When reservations are fully utilized, falls back to balanced on-demand.

Desired capacity
Specify your group size.

1

Scaling [Info](#)

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity

2

Equal or less than desired capacity

Max desired capacity

5

Equal or greater than desired capacity

Automatic scaling - optional

Choose whether to use a target tracking policy [Info](#)

You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.



No scaling policies

Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.



Target tracking scaling policy

Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

Instance maintenance policy [Info](#)

Step 3: Add a Target-Tracking Scaling Policy

Test-ASG -> Automatic scaling -> Create dynamic scaling policy -> configure:

Create dynamic scaling policy

Policy type

Target tracking scaling

Scaling policy name

Target Tracking Policy

Metric type [Info](#)

Monitored metric that determines if resource utilization is too low or high. If using EC2 metrics, consider enabling detailed monitoring for better scaling performance.

Average CPU utilization

Target value

70

Instance warmup [Info](#)

300

seconds

☐ Disable scale in to create only a scale-out policy

[Cancel](#)

[Create](#)

Step 4: Create the Companion CPU Alarm

CloudWatch Console -> Alarms -> Create alarm -> Select metric -> EC2 -> CPUUtilization -> configure

Metric

Graph

This alarm will trigger when the blue line goes above the red line for 2 datapoints within 10 minutes.

Percent

70%

39%

8.01%

CPUUtilization

Namespace

AWS/EC2

Metric name

CPUUtilization

AutoScalingGroupName

Test-ASG

Statistic

Average

Period

5 minutes

Conditions

Threshold type



Static

Use a value as a threshold



Anomaly detection

Use a band as a threshold

Whenever CPUUtilization is...

Define the alarm condition.



Greater

> threshold



Greater/Equal

>= threshold



Lower/Equal

<= threshold



Lower

< threshold

than...

Define the threshold value.

70

Must be a number.

▼ Additional configuration

Datapoints to alarm

Define the number of datapoints within the evaluation period that must be breaching to cause the alarm to go to ALARM state.

2

out of

2

Missing data treatment

ec2-high-cpu-alarm



Actions ▼

View ▼



Explore related

Investigate ▼

Details

Tags

Mute rules - new

Actions

History

Parent alarms

Details

Name

ec2-high-cpu-alarm

Type

Metric alarm

Description

No description

State

Insufficient data

Threshold

CPUUtilization > 70 for 2 datapoints within 10 minutes

Actions

Actions enabled

Last state update

2026-07-30 12:48:00 (UTC)

Namespace

AWS/EC2

Metric name

CPUUtilization

AutoScalingGroupName

Test-ASG

Statistic

Average

Period

5 minutes

Datapoints to alarm

2 out of 2

Missing data treatment

Treat missing data as missing

Percentiles with low samples

evaluate

ARN

arn:aws:cloudwatch:us-east-1:565398309825:alarm:ec2-high-cpu-alarm

► View EventBridge rule